EPM Wizard

A local-first, ChatGPT-style AI workspace for Oracle Enterprise Performance Management (EPM) implementation — with a fully IBM Cloud hosted deployment path.

What it is

EPM Wizard lets an implementation consultant build, validate, and deploy Oracle EPM artifacts — data-entry forms, business rules, cube analysis — through a conversational chat interface instead of hand-editing XML or clicking through Oracle's admin screens. It runs entirely on your machine with **docker compose up**: no hosted server, no external database, no account. It boots in Demo Mode with a deterministic local AI and a fixture Planning application, so the whole product works with zero configuration and no API key. The UI follows the IBM Carbon Design System with IBM Plex typography.

What it does

- **Conversational form & rule building** — "Create an Actuals form with level-zero descendants of Total Payroll in rows", then iterate: "move Entity to POV", "hide March", "use aliases", and finally "deploy".
- **Validation & preview** — every specification is checked against real tenant metadata (cubes, dimensions, member existence, sizing) and rendered as an interactive EPM-style grid in the chat before anything is deployed.
- **Guarded deployment** — explicit approval cards, production safeguards (a persistent PROD badge plus a typed confirmation phrase), full audit records, and post-deployment verification: a form is only marked "verified" once Oracle confirms it exists.
- **Cube Architecture visualizer** — deterministic cube maps, form coverage, cell intersections, cross-cube comparison, and sizing, derived only from real metadata.
- **Context engine** — captures tenant metadata locally with per-section honesty (complete / partial / derived / unavailable) and shares it as portable, secret-free .epwcontext packages.
- **Complete local history** — projects, conversations, artifacts, deployments, and audit records persist in local SQLite and survive restarts; export/import for backup or team sharing.

The core design principle: deterministic artifacts

EPM Wizard is more than a chatbot wrapper because the language model **never owns the deployable artifact**. The model interprets intent and proposes a structured, typed specification; deterministic application code does everything that must be correct and reproducible — validation, exact member resolution (no fuzzy substitution), safe XML rendering, byte-identical packaging with SHA-256 checksums, deployment, and verification. There is no shell execution of model output: every action maps to a typed, allowlisted backend function behind a single connector boundary.

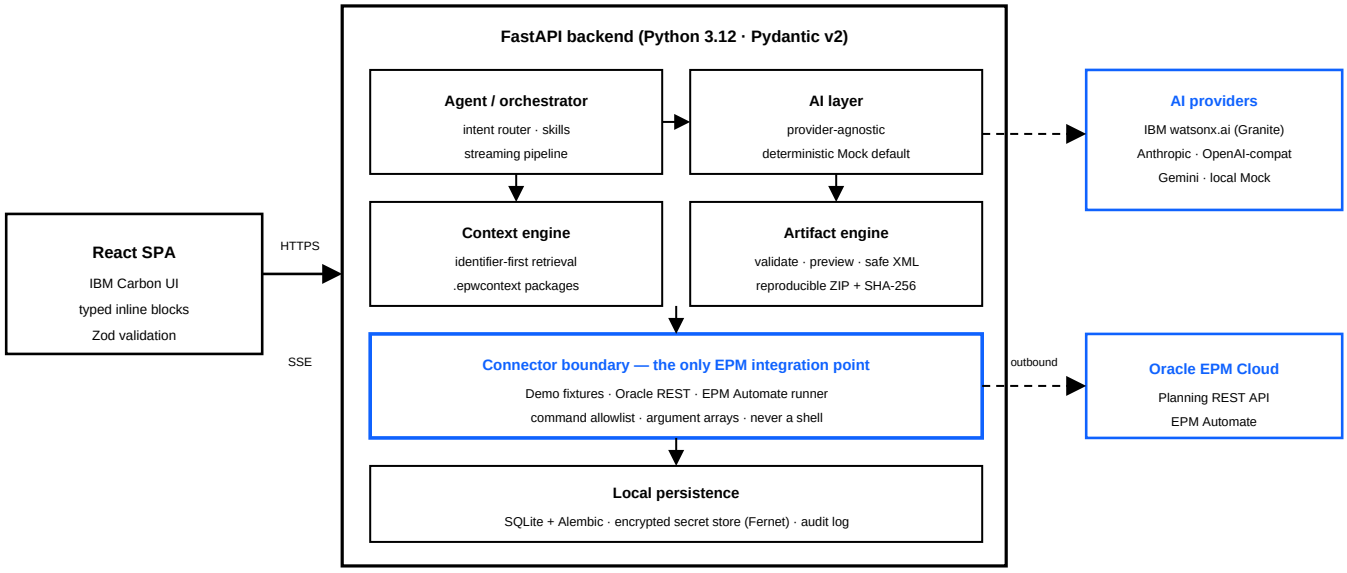
```
Intent → context retrieval → proposed spec → Pydantic validation → tenant metadata validation →
preview → user approval → deterministic generation → deploy → verify → history
```

Security posture

- Secrets never reach the model, logs, chat history, context packages, or git — a centralized redactor scrubs every log line, tool result, and diagnostics bundle.
- API keys and passwords live in a local encrypted secret store (Fernet), not the database; pasted credentials are detected, redacted before storage, and flagged to the user.
- The EPM Automate runner enforces a strict command allowlist and subprocess argument arrays — never **shell=True** — with timeouts, output redaction, and no path traversal.

Technical: system architecture

A hybrid, local-first system orchestrated by Docker Compose: a React + TypeScript SPA (Vite, IBM Carbon, TanStack Query, Zustand) talking to a FastAPI backend over HTTPS with Server-Sent Events for streaming chat. Python + Pydantic own the canonical schemas; TypeScript interfaces and Zod validators are code-generated from them, and a CI drift test fails if they diverge. Chat responses stream as typed blocks (form previews, validation reports, deployment progress, cube maps) that the client upserts in place by stable id.



The connector boundary is the single authoritative EPM integration point — the model can never call Oracle REST or EPM Automate directly, and operations are classified read-only / execution / modifying / destructive, with the latter two requiring explicit approval.

IBM Cloud: the implementation and the goal

The goal is an **all-IBM hosted topology** for teams — "email invite + link → sign in → request to cloud" — while local development stays unchanged. The plan: train an AI on the team's own EPM data on IBM Cloud, run inference on watsonx.ai, host the site on Code Engine, gate access with App ID (optionally behind an IBM VPN), and plug into Oracle EPM through the existing connector boundary. A local exporter turns validated work — conversations plus Pydantic-validated form/rule artifacts — into a fully redacted instruction-tuning corpus, so the tuned Granite model learns exactly the deterministic, validated structure the product already enforces. Training has two paths: managed **Tuning Studio** (no GPUs to manage, billed per run), or **GPU-as-a-Service** VPC instances for full fine-tunes, deprovisioned after each run. Everything is provisioned by Terraform plus a deploy script in **deploy/ibm-cloud/**, and Code Engine scales to zero when idle, keeping small-team cost near zero.

Need	IBM Cloud service	Role in EPM Wizard
AI inference	watsonx.ai (Granite models)	First-class provider type, selectable in Settings like any other
AI training	Tuning Studio / GPU-as-a-Service	Tune Granite on the redacted corpus exported from local usage
Hosting	Code Engine + Container Registry	The two existing Docker images deploy unchanged; scales to zero
Access & invites	App ID (OIDC) · optional Client-to-Site VPN	Email-invite onboarding; default is browser-only — no VPN client needed
Secrets	Secrets Manager	Oracle credentials and API keys injected as Code Engine secrets
Storage & data	Cloud Object Storage · Databases for PostgreSQL	Corpus and backups in COS; optional managed Postgres for real teams

In hosted mode the backend reaches Oracle EPM Cloud outbound over HTTPS from the VPC — no inbound exposure — and every local safeguard (validation gates, PROD confirmation phrase, audit records, post-deployment verification) applies unchanged. The database has three tiers: ephemeral demo, single-instance SQLite with scheduled backups to COS, or managed PostgreSQL with the same Alembic migrations for teams that need to scale past one instance.